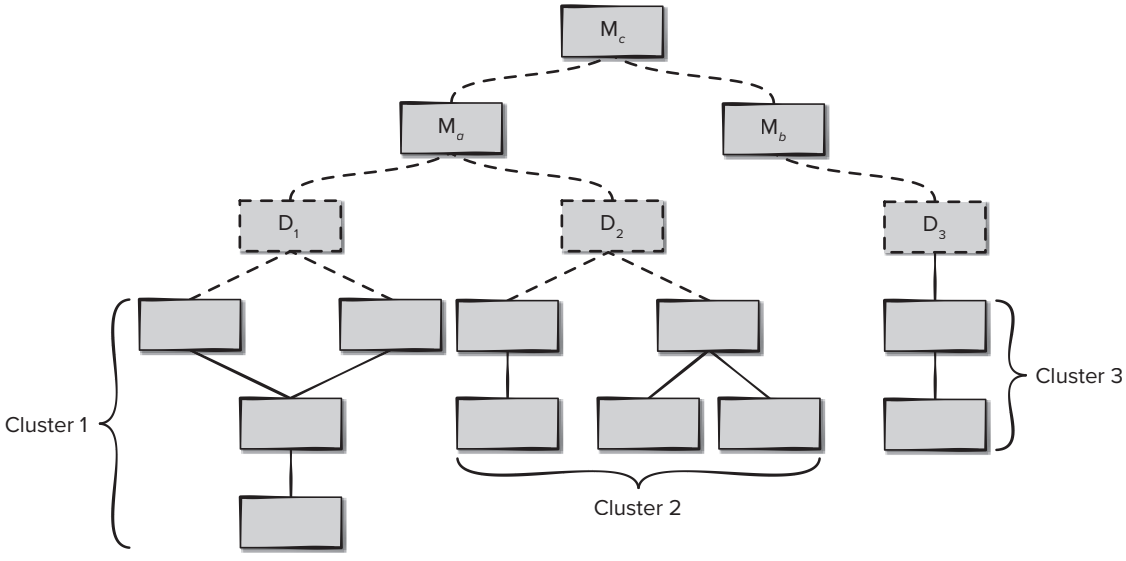
FIGURE 20.2 Bottom-up integration



Integration follows the pattern illustrated in Figure 20.2. Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to M_a. Drivers D_1 and D_2 are removed, and the clusters are interfaced directly to M_a. Similarly, driver D_3 for cluster 3 is removed prior to integration with module M_b. Both M_a and M_b will ultimately be integrated with component M_c, and so forth.

As integration moves upward, the need for separate test drivers lessens. In fact, if the top two levels of program structure are integrated top down, the number of drivers can be reduced substantially and integration of clusters is greatly simplified.

20.2.3 Continuous Integration

Continuous integration is the practice of merging components into the evolving software increment once or more each day. This is a common practice for teams following agile development practices such as XP (Section 3.5.1) or DevOps (Section 3.5.2). Integration testing must take place quickly and efficiently if a team is attempting to always have a working program in place as part of continuous delivery. It is sometimes hard to maintain systems with the use of continuous integration tools [Ste18]. Maintenance and continuous integration issues are discussed in more detail in Section 22.4.

Smoke testing is an integration testing approach that can be used when product software is developed by an agile team using short increment build times. Smoke testing might be characterized as a rolling or continuous integration strategy. The software is rebuilt (with new components added) and smoke tested every day. It is designed as a pacing mechanism for time-critical projects, allowing the software team

to assess the project on a frequent basis. In essence, the smoke-testing approach encompasses the following activities:

1. Software components that have been translated into code are integrated into a *build.* A build includes all data files, libraries, reusable modules, and engineered components that are required to implement one or more product functions.
2. A series of tests is designed to expose errors that will keep the build from properly performing its function. The intent should be to uncover "show-stopper" errors that have the highest likelihood of throwing the software project behind schedule.
3. The build is integrated with other builds, and the entire product (in its current form) is smoke tested daily. The integration approach may be top down or bottom up.

The daily frequency of testing gives both managers and practitioners a realistic assessment of integration testing progress. McConnell [McC96] describes the smoke test in the following manner:

> The smoke test should exercise the entire system from end to end. It does not have to be exhaustive, but it should be capable of exposing major problems. The smoke test should be thorough enough that if the build passes, you can assume that it is stable enough to be tested more thoroughly.

Smoke testing provides a number of benefits when it is applied on complex, time-critical software projects:

- **Integration risk is minimized.** Because smoke tests are conducted daily, incompatibilities and other showstopper errors are uncovered early, thereby reducing the likelihood of serious schedule impact when errors are uncovered.
- **The quality of the end product is improved.** Because the approach is construction (integration) oriented, smoke testing is likely to uncover functional errors as well as architectural and component-level design errors. If these errors are corrected early, better product quality will result.
- **Error diagnosis and correction are simplified.** Like all integration testing approaches, errors uncovered during smoke testing are likely to be associated with "new software increments"—that is, the software that has just been added to the build(s) is a probable cause of a newly discovered error.
- **Progress is easier to assess.** With each passing day, more of the software has been integrated and more has been demonstrated to work. This improves team morale and gives managers a good indication that progress is being made.

In some ways smoke testing resembles regression testing (discussed in Section 20.3), which helps to ensure that the newly added components do not interfere with the behaviors of existing components that were previously tested. To do this, it is a good idea to rerun a subset of the test cases that executed with the existing software component before the new components were added. The effort required to rerun test cases is not trivial, and automated testing can be used to reduce the time and effort re-created to rerun these test cases [Net18]. A complete discussion of automated testing is

beyond the scope of this chapter, but links to representative tools can be found on the Web pages that supplement this book.[2]

20.2.4 Integration Test Work Products

An overall plan for integration of the software and a description of specific tests is documented in a *test specification.* This work product incorporates a test plan and a test procedure and becomes part of the software configuration. Testing is divided into phases and incremental builds that address specific functional and behavioral characteristics of the software. For example, integration testing for the *SafeHome* security system might be divided into the following test phases: user interaction, sensor processing, communications functions, and alarm processing.

Each integration test phase delineates a broad functional category within the software and generally can be related to a specific domain within the software architecture. Therefore, software increments are created to correspond to each phase.

A schedule for integration, the development of scaffolding software (Section 19.2.1), and related topics are also discussed as part of the test plan. Start and end dates for each phase are established, and "availability windows" for unit-tested modules are defined. When developing a project schedule, you'll have to consider the manner in which integration occurs so that components will be available when needed. A brief description of scaffolding software (stubs and drivers) concentrates on characteristics that might require special effort. Finally, test environment and resources are described. Unusual hardware configurations, exotic simulators, and special test tools or techniques are a few of many topics that may also be discussed.

The detailed testing procedure that is required to accomplish the test plan is described next. The order of integration and corresponding tests at each integration step are described. A listing of all test cases (annotated for subsequent reference) and expected results are also included. In the agile world, this level of test-case description occurs when code to implement the user story is being developed so the code can be tested as soon as it is ready for integration.

A history of actual test results, problems, or peculiarities is recorded in a *test report* that can be appended to the *test specification.* It is often best to implement the test report as a shared Web document to allow all stakeholders access to the latest test results and the current state of the software increment. Information contained in this online document can be vital to developers during software maintenance (Section 4.9).

20.3 Artificial Intelligence and Regression Testing

Each time a new module is added as part of integration testing, the software changes. New data flow paths are established, new input/output (I/O) may occur, and new control logic is invoked. Side effects associated with these changes may cause problems with functions that previously worked flawlessly. In the context of an integration

2 See the SEPA 9e website.

test strategy, *regression testing* is the reexecution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects. Regression tests should be executed every time a major change is made to the software (including the integration of new components). Regression testing helps to ensure that changes (due to testing or for other reasons) do not introduce unintended behavior or additional errors.

Regression testing may be conducted manually, by reexecuting a subset of all test cases or using automated capture/playback tools. *Capture/playback tools* enable the software engineer to capture test cases and results for subsequent playback and comparison. The *regression test suite* (the subset of tests to be executed) contains three different classes of test cases:

- A representative sample of tests that will exercise all software functions
- Additional tests that focus on software functions that are likely to be affected by the change
- Tests that focus on the software components that have been changed

As integration testing proceeds, the number of regression tests can grow quite large. Therefore, the regression test suite should be designed to include only those tests that address one or more classes of errors in each of the major program functions.

Yoo and Harman [Yoo13] write about potential uses of artificial intelligence (AI) in identifying test cases for use in regression test suites. A software tool could examine the dependencies among the components in the software increment after the new components have been added and generate test cases automatically to use for regression testing. Another possibility would be using machine learning techniques to select sets of test cases that will optimize the discovery of component collaboration errors. This work is promising, but still requires significant human interaction to review the test cases and the recommended order for executing them.

SAFEHOME



Regression Testing

The scene: Doug Miller's office, as integration testing is under way.

The players: Doug Miller, software engineering manager; Vinod, Jamie, Ed, and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Doug: It seems to me that we are not spending enough time retesting software components after new components are integrated.

Vinod: I guess that is true, but isn't it good enough that we are testing the new components' interactions with the components they are supposed to collaborate with?

Doug: Not always. Sometimes components make unintended change to data used by other components. I know we are busy, but it is important to discover these problems early.

Shakira: We do have a test-case repository we have been drawing from. Perhaps we can randomly select several test cases to run using our automated testing framework.

Doug: That's a start. But maybe we should be more strategic in how we select our test cases.

Ed: I suppose we could use our test-case/requirement traceability table and check our CRC card model.

Vinod: I have been using continuous integration, meaning I integrate each component as soon as one of the developers passes it to me. I try to run a series of regression tests on the partially integrated program.

Jamie: I've been trying to design a set of appropriate tests for each function in the system. Maybe I should tag some of the more important ones for Vinod to use for regression testing.

Doug (to Vinod): How often will you run the regression test cases?

Vinod: Every day I integrate a new component I will use the regression test cases . . . until we decide the software increment is done.

Doug: Let's try using Jamie's regression test cases as they are created and see how things go.

20.4 Integration Testing in the OO Context

Object-oriented software does not have an obvious hierarchical control structure, so traditional top-down and bottom-up integration strategies (Section 20.2) have little meaning. In addition, integrating operations one at a time into a class (the conventional incremental integration approach) is often impossible because of the "direct and indirect interactions of the components that make up the class" [Ber93].

There are two different strategies for integration testing of OO systems: thread-based testing and use-based testing [Bin99]. The first, *thread-based testing*, integrates the set of classes required to respond to one input or event for the system. Each thread is integrated and tested individually. Regression testing is applied to ensure that no side effects occur. An important strategy for integration testing of OO software is thread-based testing. Threads are sets of classes that respond to an input or event.

The second integration approach, *use-based testing,* begins the construction of the system by testing those classes (called *independent classes*) that use very few (if any) *server* classes. After the independent classes are tested, the next layer of classes, called *dependent classes,* that use the independent classes are tested. This sequence of testing layers of dependent classes continues until the entire system is constructed. Use-based tests focus on classes that do not collaborate heavily with other classes.

The use of scaffolding software also changes when integration testing of OO systems is conducted. Drivers can be used to test operations at the lowest level and for the testing of whole groups of classes. A driver can also be used to replace the user interface so that tests of system functionality can be conducted prior to implementation of the interface. Stubs can be used in situations in which collaboration between classes is required but one or more of the collaborating classes has not yet been fully implemented.

Cluster testing is one step in the integration testing of OO software. Here, a cluster of collaborating classes (determined by examining the CRC and object-relationship model) is exercised by designing test cases that attempt to uncover errors in the collaborations.

20.4.1 Fault-Based Test-Case Design[3]

The object of *fault-based testing* within an OO system is to design tests that have a high likelihood of uncovering plausible faults. Because the product or system must conform to customer requirements, preliminary planning required to perform fault-based testing begins with the analysis model. The strategy for fault-based testing is to hypothesize a set of plausible faults and then derive tests to prove each hypothesis. The tester looks for plausible faults (i.e., aspects of the implementation of the system that may result in defects). To determine whether these faults exist, test cases are designed to exercise the design or code.

Of course, the effectiveness of these techniques depends on how testers perceive a plausible fault. If real faults in an OO system are perceived to be implausible, then this approach is really no better than any random testing technique. However, if the analysis and design models can provide insight into what is likely to go wrong, then fault-based testing can find significant numbers of errors with relatively low expenditures of effort.

Integration testing looks for plausible faults in operation calls or message connections. Three types of faults are encountered in this context: unexpected result, wrong operation/message used, and incorrect invocation. To determine plausible faults as functions (operations) are invoked, the behavior of the operation must be examined.

Integration testing applies to attributes as well as to operations. The "behaviors" of an object are defined by the values that its attributes are assigned. Testing should exercise the attributes to determine whether proper values occur for distinct types of object behavior.

It is important to note that integration testing attempts to find errors in the client object, not the server. Stated in conventional terms, the focus of integration testing is to determine whether errors exist in the calling code, not the called code. The operation call is used as a clue, a way to find test requirements that exercise the calling code.

The approach for multiple-class partition testing is similar to the approach used for partition testing of individual classes. A single class is partitioned as discussed in Section 19.6.1. However, the test sequence is expanded to include those operations that are invoked via messages to collaborating classes. An alternative approach partitions tests based on the interfaces to a particular class. Referring to Figure 20.3, the **Bank** class receives messages from the **ATM** and **Cashier** classes. The methods within **Bank** can therefore be tested by partitioning them into those that serve **ATM** and those that serve **Cashier.**

Kirani and Tsai [Kir94] suggest the following sequence of steps to generate multiple-class random test cases:

1. For each client class, use the list of class operations to generate a series of random test sequences. The operations will send messages to other server classes.
2. For each message that is generated, determine the collaborator class and the corresponding operation in the server object.

3 Sections 20.4.1 and 20.4.2 have been adapted from an article by Brian Marick originally posted on the Internet newsgroup comp.testing. This adaptation is included with the permission of the author. For further information on these topics, see [Mar94]. It should be noted that the techniques discussed in Sections 20.4.1 and 20.4.2 are also applicable for conventional software.